Bongsudae

A BMP collector written in Go

The Bongsudae was a beacon tower of old Korea, relaying news from hill to hill.

This one relays news from your routers.



2026-11 · v0.1 · open source (ISC) · github.com/yunsog/bongsudae

Why a new collector

Problems we had running openbmpd

29

restarts in one month

One internet scanner connection ended the whole collector. After each restart 24 routers resent their full tables, leaving monitoring blind

93 GB

of container logs

The same error logged without end. No log rotation, so the disk filled up

0 routes

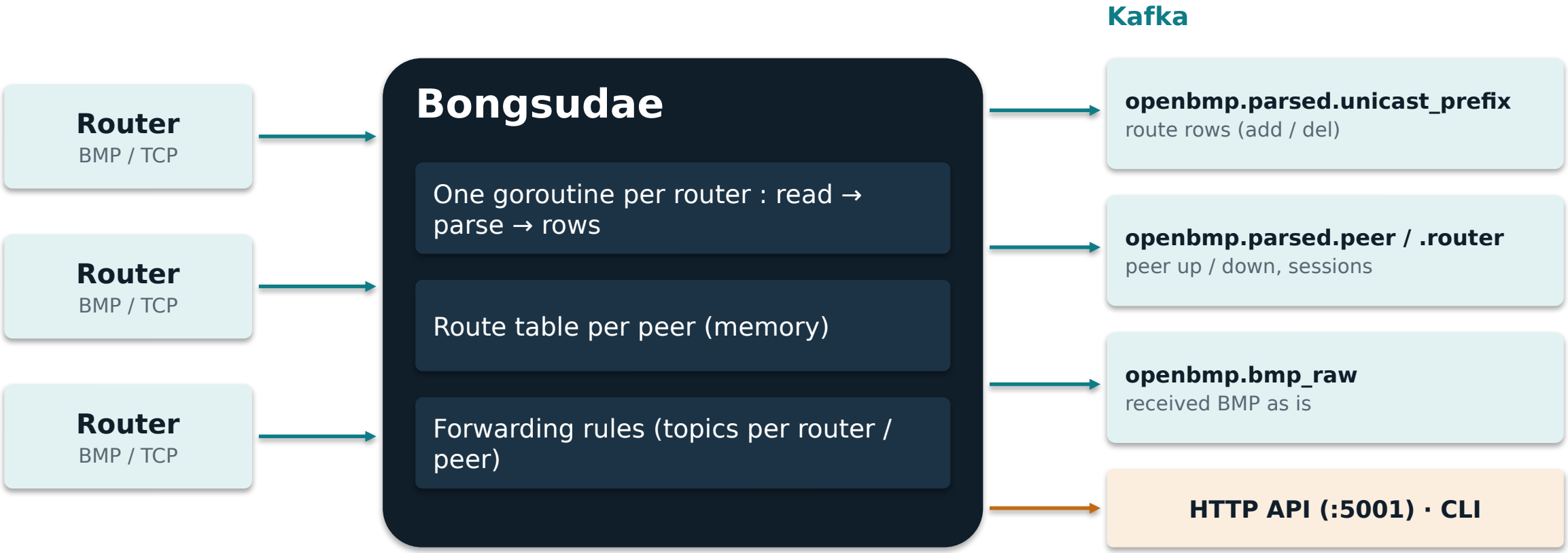
from one router

One RFC-violating Peer Up (no OPEN messages) from one router made it drop that router's whole session

openbmpd (the OpenBMP collector) has had no release since 2022-10: nobody to fix it

Architecture

What Bongsudae does



Kafka consumers (hijack monitoring, search stores ...) read the same topics in the same format as with openbmpd

Features

Four key points

1 Drop-in compatible

Same OpenBMP Message Bus API v1.7 (TSV) format, so consumer code stays. openbmpd's bugs fixed per the RFCs

2 Collection keeps running

A bad connection or message is dropped alone. Kafka trouble: errors summarised, restart on its own

3 Route tables and HTTP API

Live routes of every peer in memory: status, prefix lookup and session reset over the API

4 Multi-core

Routers processed in parallel. Full tables are taken in quickly after a restart

Comparison

Compared with other BMP collectors

	Bongsudae	openbmpd	goBMP	pmacct (pmbmpd)
Language	**Go**	C++	Go	C
Kafka messages	**OpenBMP v1.7 (TSV)**	OpenBMP v1.7 (TSV)	JSON	JSON / Avro
openbmpd consumers work as is	**Yes (checked row by row)**	(original)	No	No
Raw BMP output (bmp_raw)	**Yes**	Yes	Yes (raw mode)	No
Address families	**IPv4 / IPv6 unicast**	unicast, L3VPN, EVPN, BGP-LS	unicast, VPN, EVPN, BGP-LS, SR Policy, FlowSpec	Many
Route table in the collector	**Yes (HTTP API queries)**	No (DB consumer builds it)	No (forwards only)	Yes (periodic dumps)
License	**ISC**	EPL-1.0 / BSD	Apache-2.0	GPL
Latest release	**0.1 (2026-11)**	None since 2022-10 (2.2.3)	Active	Active

From each project's public documents and repository, 2026-10. Bongsudae covers fewer address families but is message-compatible with openbmpd, with its bugs fixed

Drop-in compatible

Same input, same rows : checked row by row

1 → 2 → 3

Same input

The raw BMP openbmpd received (bmp_raw) is fed to Bongsudae too

Two collectors

openbmpd and Bongsudae each write to Kafka (production / test topics)

Row compare

Rows paired on all 32 fields except time / seq. Every leftover row explained

- **Same format :** same topics, field order and text forms, so consumer code stays
- **Bugs fixed :** 6 openbmpd message bugs follow the RFCs. e.g. AS_SET {a,b} counts as 2 ASes in openbmpd, 1 in Bongsudae (RFC 4271)
- **Everything else identical :** with the fixes applied, 0 differing rows (1 hour, 4.76 M UPDATEs)

10,901,264

/ 10,901,415 rows identical (80-minute sample). The rest are window edges

9 hours

live side-by-side compare, 7 ~ 9 M rows per hour, 0 unexplained differences

Collection keeps running

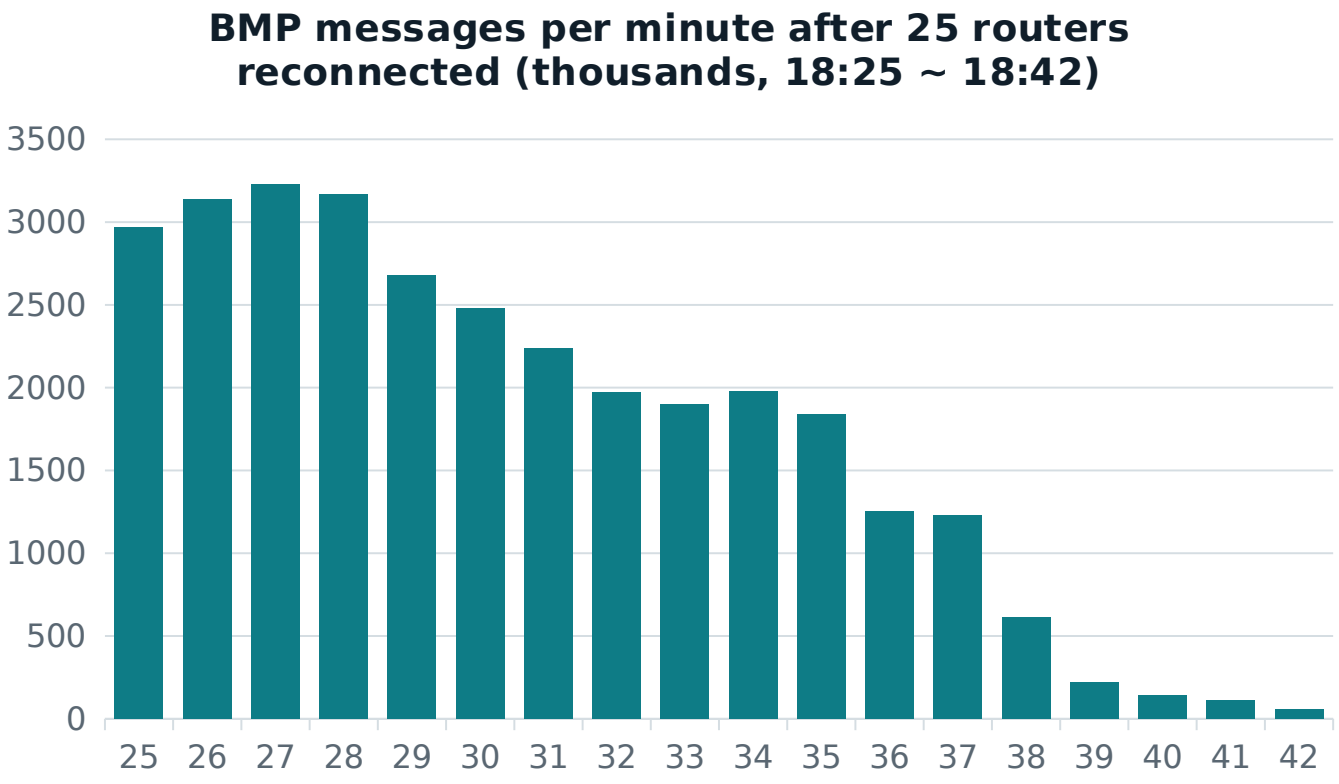
A problem stays where it happens

Situation	openbmpd	Bongsudae
Internet scanner connects to the BMP port	Whole collector ends	Outside the allow list: closed at once. No Initiation in 30 s: that connection only
RFC-violating message (Peer Up without OPEN)	Drops the router's whole session	Session kept, peer up with empty capabilities, one log line
The same error repeats	Logged without end (93 GB of logs)	Summarised to one line per kind per minute (max 100 per router)
Broken message, code bug (panic)	-	Only that message is dropped
Kafka restart (tens of seconds down)	-	Reconnects on its own. Seen in production: 0 send failures
Config file mistake (typo, bad prefix)	-	Does not start. On reload while running: nothing changes

- : not seen or tested with openbmpd

In production

Switch-over : full tables back in 18 min



Switch on 2026-10-30, 1-minute stats. Bound by router send speed (max 3.2 M per minute)

25 / 25 routers

including 180 k routes from the router openbmpd always dropped (one RFC-violating message)

77 M routes

live route tables of 1,328 peers (IPv4 55 M + IPv6 22 M), about 12 GB of memory

0

errors and Kafka send failures (25 hours after the switch, 161 M BMP messages)

Route tables in memory, ready to query

- **Status :** router / peer state, route counts, warnings, capabilities, history
- **Prefix lookup :** exact / longest / covering / more
- **Hijack check :** routes inside our prefixes whose origin is not our AS (not_origin)
- **Many at once :** 5000 registered prefixes in one request
- **CLI = API :** arguments map 1:1 to API paths, other servers with -api

```
$ bongsudae -status prefix p=192.0.2.0/22 \
      match=more not_origin=64500
192.0.2.0/22  match=more  not_origin=[64500]  1 / 1
rows

ROUTER       PEER         PREFIX       AS PATH
198.51.100.7 203.0.113.9  192.0.2.0/24  64511 64999

$ curl :5001/api/routers/198.51.100.7
{"state": "up", "peers_up": 12,
 "peers_synced": 12, "routes_v4": 1204513, ...}
```

Addresses and ASes in the example are documentation values (RFC 5737, RFC 5398)

Operations

What operations need

1	**Session reset**	Closing a router's BMP session via the API makes it reconnect and resend everything. Progress from End-of-RIB (peers_synced)
2	**Kafka forwarding rules**	Choose topics, or none, per router / peer to spread consumer load. Saved to a file, kept across restarts
3	**bmp_raw output**	Received BMP to Kafka unchanged. Feed another collector, or shadow-test a new release on live input
4	**Changes need a key**	Queries are open; rule changes / resets need a key and log the caller (X-Actor)
5	**Config reload**	systemctl reload : allow lists and bmp_raw change without a restart. All checked first; on error nothing changes

Upgrades

Each release is tested on full tables first

1 Save full tables

The full tables routers resend at a production restart (raw BMP) go to a file

2 Side by side

Same input to the running and the new release. Outputs streamed and compared (no files)

3 Row by row

Rows bound for Kafka compared down to seq. Differences shown per field with examples

```
$ tools/regress.sh
unicast_prefix : records 34,335,279, rows 84,199,037, differing 0
router : rows 25, differing 0
peer   : rows 1,328, differing 0
verdict : same (every row identical, seq included)
```

84.2 M rows

next and running release give identical output (full tables, every release)

Any difference must be an intended fix before the upgrade. A shadow run reading production bmp_raw also compares live

Build

Small and simple

1

binary. Install = copy one file (no container)

1

external library (Kafka client franz-go)

7,200 lines

of Go code (tests and tools included)

0

race detector warnings (load + concurrent API calls)

Operation : systemd service + one config file (no options, systemctl reload re-reads it)

```
# /etc/bongsudae/bongsudae.conf
[bmp]
listen = :5000
allow  = 192.0.2.0/24,     # ends with a comma :
         198.51.100.0/24   # continues here

[kafka]
prefix = openbmp.parsed
[http]
listen = :5001
allow  =                   # empty = everyone
```

Three inputs : tcp (production) · kafka (shadow test reading bmp_raw) · file (dump replay, release compare)

Roadmap

In production, now open source

Now

Released

Next

In production

Replaced openbmpd; routes that never arrived now do. systemd service + config file

Open source v0.1

GitHub yunsog/bongsudae, ISC license. Source + English docs (manual, API, Kafka messages)

To add

Upgrades without reconnects, even Kafka partitions (split big peers), optional JSON output, VPN / EVPN

Bongsudae

Drop-in compatible · Keeps collecting · Route tables and API

A BMP collector in Go · OpenBMP v1.7 compatible messages · ISC-licensed open source

https://github.com/yunsog/bongsudae